



3° MINI-CONTEST PRESENTATION

CORSO DI MACHINE LEARNING 22/23

Danilo Persico
Matr. M63001163

TABLE OF CONTENTS

The following content will be presented:

- Dataset and mini-contest description;
- Model implementation;
- Model testing;

DATASET AND MINI-CONTEST DESCRIPTION



DATASET AND MINI-CONTEST DESCRIPTION

The dataset consists of features extracted from various frames of videos depicting different types of cones used in Formula SAE races.

These types are:

- Big orange cones, delimiting the beginning and ending of the track
- Little orange cones, delimiting the finish area
- Blue cones, delimiting the right border of the track
- Yellow cones, delimiting the left border of the track



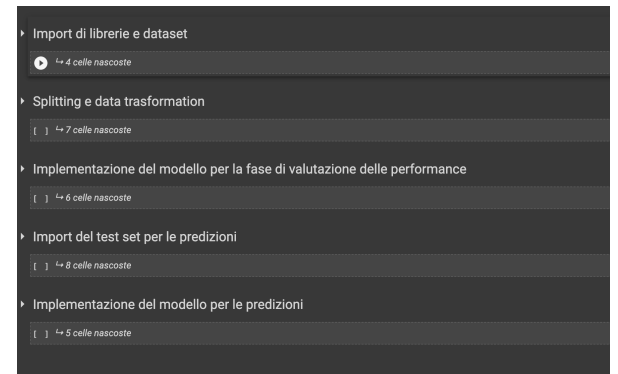
DATASET AND MINI-CONTEST DESCRIPTION

The aim of the third mini-contest is to properly classify a specific cone detected by sensors given its extracted features.

A label is assigned to each type of cone, going from 1 to 4.



MODEL IMPLEMENTATION



MODEL IMPLEMENTATION

```
Import di librerie e dataset

[1] 1 import numpy as np
    2 import pandas as pd
    3 from keras.models import Sequential
    4 from keras.layers import Dense
    5 import tensorflow as tf
    6 import matplotlib.pyplot as plt
    7 import imblearn
    8 from sklearn.preprocessing import LabelEncoder
    9 from collections import Counter
   10 from imblearn.over_sampling import SMOTE

[2] 1 from google.colab import drive
    2 drive.mount('/content/drive')

Mounted at /content/drive

[3] 1 dataset = pd.read_csv('/content/drive/MyDrive/mergedResNet34TrainingCONCAT.csv')
    2
    3 dataset.head(2)
```

	Unnamed: 0	Unnamed: 0.1	f0	f1	f2	f3	f4	f5	f6	f7	...	f505	f506	f507	f508	f509	f510	f511	images	cone_name	label
0	0	0	0.878583	0.798011	1.032221	1.025648	0.932683	0.942697	1.145143	0.899828	...	1.018030	0.716998	0.831250	0.967619	0.828388	0.796302	0.869872	cone35_frame0046_0	cone35_frame0046_0	2
1	1	1	0.789336	0.987613	0.985949	0.994959	0.855062	0.964386	1.030023	0.886103	...	1.152784	0.850843	0.912317	0.947074	0.869836	0.876474	0.877852	cone35_frame0046_1	cone35_frame0046_1	2

2 rows x 517 columns

```
[4] 1 dataset1 = dataset.drop(['Unnamed: 0', 'Unnamed: 0.1', 'images', 'cone_name'], axis=1)
    2
    3 dataset1.head(2)
```

	f0	f1	f2	f3	f4	f5	f6	f7	f8	f9	...	f503	f504	f505	f506	f507	f508	f509	f510	f511	label
0	0.878583	0.798011	1.032221	1.025648	0.932683	0.942697	1.145143	0.899828	1.105560	0.982394	...	0.940622	0.848088	1.018030	0.716998	0.831250	0.967619	0.828388	0.796302	0.869872	2
1	0.789336	0.987613	0.985949	0.994959	0.855062	0.964386	1.030023	0.886103	0.992618	1.063324	...	0.875840	0.851216	1.152784	0.850843	0.912317	0.947074	0.869836	0.876474	0.877852	2

2 rows x 513 columns

MODEL IMPLEMENTATION

```
Splitting e data trasformation

[5] 1 #Splitting 80-20, dove 20 è il test set
    2 firstsplit = dataset1.sample(frac = 0.80, random_state = 100)
    3 test = dataset1.drop(firstsplit.index)
    4
    5 #Splitting del restante 80 in 75-25, dove 75 è il training set e 25 il validation set
    6 train = firstsplit.sample(frac = 0.75, random_state = 100)
    7 validation = firstsplit.drop(train.index)
    8
    9 print(len(train))
   10 print(len(test))
   11 print(len(validation))

6702
2234
2234

[6] 1 #Rimozione dai tre set della label
    2
    3 trainlabel = train.pop('label')
    4 testlabel = test.pop('label')
    5 validlabel = validation.pop('label')
    6
    7 print(trainlabel.shape, testlabel.shape, validlabel.shape)

(6702,) (2234,) (2234,)
```


MODEL IMPLEMENTATION

```
[7] 1 #Check sulla percentuale di distribuzione delle classi nel training set
2
3 trainlabel = LabelEncoder().fit_transform(trainlabel)
4
5 counter = Counter(trainlabel)
6
7 for k,v in counter.items():
8     per = v / len(trainlabel) * 100
9     print('Class=%d, n=%d (%.3f%%)' % (k+1, v, per))
```

```
Class=1, n=927 (13.832%)
Class=4, n=2519 (37.586%)
Class=2, n=2340 (34.915%)
Class=3, n=916 (13.668%)
```

```
[8] 1 #Applicazione dello SMOTE sul training set
2
3 oversample = SMOTE()
4
5 train, trainlabel = oversample.fit_resample(train, trainlabel)
6
7 counter = Counter(trainlabel)
8
9 for k,v in counter.items():
10     per = v / len(trainlabel) * 100
11     print('Class=%d, n=%d (%.3f%%)' % (k+1, v, per))
```

```
Class=1, n=2519 (25.000%)
Class=4, n=2519 (25.000%)
Class=2, n=2519 (25.000%)
Class=3, n=2519 (25.000%)
```

```
[9] 1 x = train.values
2
3 x_val = validation.values
4
5 print(x, x_val)
```

```
[10] 1 validlabel = LabelEncoder().fit_transform(validlabel)
2
3 x_val_lbl = validlabel
4
5 x_val_lbl1 = pd.get_dummies(x_val_lbl)
6
7 x_val_lbl2 = x_val_lbl1.values
```

```
[11] 1 x_lbl = trainlabel
2
3 x_lbl1 = pd.get_dummies(x_lbl)
4
5 x_lbl2 = x_lbl1.values
```

MODEL IMPLEMENTATION

▼ Implementazione del modello per la fase di valutazione delle performance

```
[12] 1 def modelML():  
      2     model = Sequential()  
      3  
      4     model.add(Dense(512, input_dim = 512, activation = 'relu'))  
      5  
      6     model.add(tf.keras.layers.Dropout(0.3))  
      7  
      8     model.add(Dense(4, activation = 'softmax'))  
      9  
     10     optimizer = tf.keras.optimizers.SGD(learning_rate=0.001,momentum=0.7)  
     11  
     12     model.compile(loss = 'categorical_crossentropy', optimizer = optimizer, metrics = ['accuracy'])  
     13  
     14     return model
```

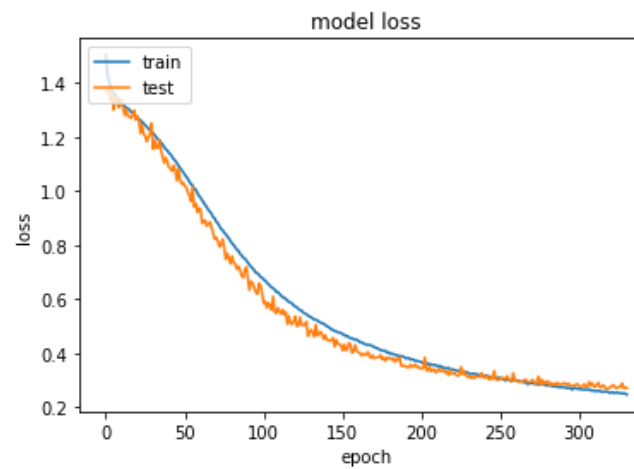
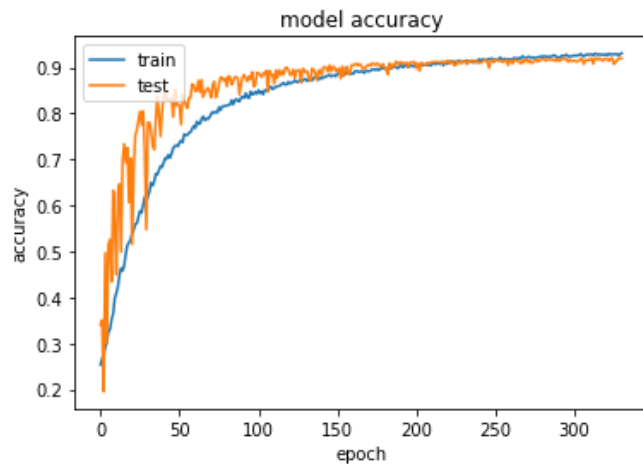
MODEL IMPLEMENTATION

```
[57] 1 modelMLCall = modelML()
      2
      3 import time
      4 class SOMT(tf.keras.callbacks.Callback):
      5     def __init__(self, model, train_thold, valid_thold):
      6         super(SOMT, self).__init__()
      7         self.model=model
      8         self.train_thold=train_thold
      9         self.valid_thold=valid_thold
      10
      11     def on_train_begin(self, logs=None):
      12         print('Starting Training - training will halt if training accuracy achieves or exceeds ', self.train_thold)
      13         print('and validation accuracy meets or exceeds ', self.valid_thold)
      14         msg='{0:~8s}{1:~12s}{2:~12s}{3:~12s}{4:~12s}{5:~12s}'.format('Epoch', 'Train Acc', 'Train Loss', 'Valid Acc', 'Valid Loss', 'Duration')
      15         print(msg)
      16
      17     def on_train_batch_end(self, batch, logs=None):
      18         acc=logs.get('accuracy')* 100 # get training accuracy
      19         loss=logs.get('loss')
      20         msg='{0:~1s}processed batch {1:~4s} training accuracy= {2:~8.3f} loss: {3:~8.5f}'.format(' ', str(batch), acc, loss)
      21         print(msg, '\r', end='')
      22
      23     def on_epoch_begin(self, epoch, logs=None):
      24         self.now= time.time()
      25
      26     def on_epoch_end(self, epoch, logs=None):
      27         later=time.time()
      28         duration=later-self.now
      29         tacc=logs.get('accuracy')
      30         vacc=logs.get('val_accuracy')
      31         tr_loss=logs.get('loss')
      32         v_loss=logs.get('val_loss')
      33         ep=epoch+1
      34         print(f'{ep:~8.0f} {tacc:~12.2f}{tr_loss:~12.4f}{vacc:~12.2f}{v_loss:~12.4f}{duration:~12.2f}')
      35         if tacc>= self.train_thold and vacc>= self.valid_thold:
      36             print( f'\ntraining accuracy and validation accuracy reached the thresholds on epoch {epoch + 1}' )
      37             self.model.stop_training = True # stop training
      38
      39
      40 train_thold= .93
      41 valid_thold=.91
      42 callbacks=[SOMT(modelMLCall, train_thold, valid_thold)]
      43
      44
      45 fitting = modelMLCall.fit(x, x_lb12, validation_data = (x_val, x_val_lb12), batch_size=128, epochs = 500, callbacks=callbacks, verbose=0)
```

MODEL IMPLEMENTATION

```
277 0.92 0.2765 0.92 0.2803 0.67
278 0.93 0.2689 0.92 0.2888 0.68
279 0.92 0.2721 0.91 0.2909 0.64
280 0.93 0.2721 0.91 0.2847 0.67
281 0.93 0.2682 0.91 0.2882 0.69
282 0.92 0.2720 0.91 0.2848 0.68
283 0.93 0.2693 0.91 0.2927 0.68
284 0.93 0.2677 0.92 0.2809 0.69
285 0.93 0.2701 0.92 0.2829 0.66
286 0.93 0.2676 0.91 0.2787 0.70
287 0.93 0.2676 0.92 0.2959 0.69
288 0.93 0.2659 0.91 0.2857 0.65
289 0.93 0.2664 0.92 0.2799 0.67
290 0.93 0.2640 0.91 0.2819 0.69
291 0.93 0.2634 0.92 0.2851 0.64
292 0.93 0.2643 0.92 0.2867 0.69
293 0.93 0.2631 0.92 0.2728 0.68
294 0.92 0.2628 0.92 0.2768 0.74
295 0.93 0.2641 0.92 0.2755 0.68
296 0.93 0.2604 0.92 0.2825 0.70
297 0.93 0.2574 0.91 0.2777 0.68
298 0.93 0.2616 0.91 0.2795 0.68
299 0.93 0.2575 0.91 0.2892 0.69
```

training accuracy and validation accuracy reached the thresholds on epoch 299



MODEL IMPLEMENTATION

```
[36] 1 y_testlabel = LabelEncoder().fit_transform(testlabel)
      2 y_dummylabel = pd.get_dummies(y_testlabel)
      3 y_lblvalues = y_dummylabel.values
      4
      5 y = test.values
      6
      7
      8
      9 testeval = modelMLCall.evaluate(y,y_lblvalues)
```

```
70/70 [=====] - 0s 3ms/step - loss: 0.2378 - accuracy: 0.9252
```

MODEL TESTING



MODEL TESTING

```
Import del test set per le predizioni

[40] 1 testSet = pd.read_csv('/content/drive/MyDrive/TEST.csv', sep = ',')
2
3 testSet.head(2)
```

	Unnamed: 0	f0	f1	f2	f3	f4	f5	f6	f7	f8	...	f503	f504	f505	f506	f507	f508	f509	f510	f511	image
0	0	0.911863	0.952458	1.085506	0.910698	0.880114	1.016642	0.934292	1.024749	1.228008	...	0.874452	0.974516	1.105526	0.824213	0.847534	1.071275	0.944886	0.901812	0.883428	img_1
1	1	0.843607	0.855531	1.043309	0.820285	0.940399	0.884501	0.862858	0.809213	1.116383	...	0.928182	0.858758	1.086330	0.816912	0.809086	1.059228	1.033703	1.001865	0.824272	img_2

2 rows x 514 columns

```
[41] 1 testdrop = testSet.drop(['Unnamed: 0'], axis=1)
2
3 testdrop.head(3)
```

	f0	f1	f2	f3	f4	f5	f6	f7	f8	f9	...	f503	f504	f505	f506	f507	f508	f509	f510	f511	image
0	0.911863	0.952458	1.085506	0.910698	0.880114	1.016642	0.934292	1.024749	1.228008	0.951209	...	0.874452	0.974516	1.105526	0.824213	0.847534	1.071275	0.944886	0.901812	0.883428	img_1
1	0.843607	0.855531	1.043309	0.820285	0.940399	0.884501	0.862858	0.809213	1.116383	1.116935	...	0.928182	0.858758	1.086330	0.816912	0.809086	1.059228	1.033703	1.001865	0.824272	img_2
2	0.916222	0.942230	1.002602	1.082436	0.911001	1.070029	1.015654	0.989635	1.155890	0.921690	...	0.979656	0.841254	0.963832	0.742647	0.816151	0.949110	0.962450	0.944196	0.901135	img_3

3 rows x 513 columns

```
[42] 1 testimg = testdrop.pop('image')
2
3 testimg.head(3)
```

	img_1	img_2	img_3
0	img_1	img_2	img_3
1	img_1	img_2	img_3
2	img_1	img_2	img_3

Name: image, dtype: object

```
[44] 1 datalabel = dataset1.pop('label')

[47] 1 xdata = dataset1.copy()
2
3 xdata.head(2)
```

	f0	f1	f2	f3	f4	f5	f6	f7	f8	f9	...	f502	f503	f504	f505	f506	f507	f508	f509	f510	f511
0	0.878583	0.798011	1.032221	1.025648	0.932683	0.942697	1.145143	0.899828	1.105560	0.982394	...	0.799881	0.940622	0.848088	1.018030	0.716998	0.831250	0.967619	0.828388	0.796302	0.869872
1	0.789336	0.987613	0.985949	0.994959	0.855062	0.964386	1.030023	0.886103	0.992618	1.063324	...	0.838019	0.875840	0.851216	1.152784	0.850843	0.912317	0.947074	0.869836	0.876474	0.877852

2 rows x 512 columns

```
[48] 1 ydata = datalabel.copy()
2
3 ydata.head(2)
```

	2	2
0	2	2
1	2	2

Name: label, dtype: int64

```
[49] 1 xdata1 = xdata.values
2 ydata = pd.get_dummies(ydata)
3 ydata1 = ydata.values
```

MODEL TESTING

▼ Implementazione del modello per le predizioni

```
[61] 1 ## NELLA SUB 12 CHIAMO IL MODELML2() ESCLUDENDO IL DROPOUT
```

```
[53] 1 def modelML2():  
2     model = Sequential()  
3  
4     model.add(Dense(512, input_dim = 512, activation = 'relu'))  
5  
6     #model.add(tf.keras.layers.Dropout(0.3))  
7  
8     model.add(Dense(4, activation = 'softmax'))  
9  
10    optimizer = tf.keras.optimizers.SGD(learning_rate=0.001,momentum=0.7)  
11  
12    model.compile(loss = 'categorical_crossentropy', optimizer = optimizer, metrics = ['accuracy'])  
13  
14    return model  
15  
16  
17 modelMLCall2 = modelML2()  
18
```


MODEL TESTING

```
17 modelMLCall2 = modelML2()
18
19
20 import time
21 class SOMT(tf.keras.callbacks.Callback):
22     def __init__(self, model, loss_thold):
23         super(SOMT, self).__init__()
24         self.model=model
25         self.loss_thold=loss_thold
26
27     def on_train_begin(self, logs=None):
28         print('Starting Training - training will halt if training loss goes below ', self.loss_thold)
29         msg='{0:^8s}{1:^12s}{2:^12s}{3:^12s}'.format('Epoch', 'Train Acc', 'Train Loss','Duration')
30         print (msg)
31
32     def on_train_batch_end(self, batch, logs=None):
33         acc=logs.get('accuracy')* 100 # get training accuracy
34         loss=logs.get('loss')
35         msg='{0:1s}processed batch {1:4s} training accuracy= {2:8.3f} loss: {3:8.5f}'.format(' ', str(batch), acc, loss)
36         print(msg, '\r', end='') # prints over on the same line to show running batch count
37
38     def on_epoch_begin(self,epoch, logs=None):
39         self.now= time.time()
40
41     def on_epoch_end(self,epoch, logs=None):
42         later=time.time()
43         duration=later-self.now
44         tacc=logs.get('accuracy')
45         tr_loss=logs.get('loss')
46         ep=epoch+1
47         print(f'{ep:^8.0f} {tacc:^12.2f}{tr_loss:^12.4f}{duration:^12.2f}')
48         if tr_loss<= self.loss_thold:
49             print( f'\nTraining loss went below the threshold of 0.205 on epoch {epoch + 1}' )
50             self.model.stop_training = True # stop training
51
52
53 loss_thold = .205
54 callbacks=[SOMT(modelMLCall2, loss_thold)]
55
56
57
58 fitting2 = modelMLCall2.fit(xdata1, ydata1, batch_size=128, epochs = 500, callbacks=callbacks, verbose=0)
```

MODEL TESTING

399	0.94	0.2086	0.60
400	0.94	0.2099	0.60
401	0.94	0.2083	0.61
402	0.94	0.2084	0.62
403	0.94	0.2087	0.63
404	0.94	0.2078	0.62
405	0.94	0.2059	0.63
406	0.94	0.2085	0.63
407	0.94	0.2067	0.61
408	0.94	0.2063	0.62
409	0.94	0.2064	0.61
410	0.94	0.2062	0.62
411	0.94	0.2061	0.62
412	0.94	0.2051	0.60
413	0.94	0.2081	0.60
414	0.94	0.2044	0.61

Training loss went below the threshold of 0.205 on epoch 414

MODEL TESTING

```
✓ [54] 1 prediction = modelMLCall12.predict(testdrop)
      2
      3
      4 prediction_rounded = [np.round(x) for x in prediction]
      5
      6
      7 prediction_class = [np.argmax(x) + 1 for x in prediction_rounded]
      8 print(prediction_class)

199/199 [=====] - 0s 2ms/step
[4, 4, 2, 1, 3, 1, 1, 1, 1, 1, 1, 4, 4, 1, 1, 4, 2, 2, 4, 4, 2, 2, 4, 4, 2, 2, 2, 4, 4, 4, 2,
```

```
✓ [55] 1 submission = pd.DataFrame({'ID':testImg,'Class':prediction_class})
      2
      3 print(submission.head(5))
      4
      5 print(submission.shape)

      ID  Class
0  img_1     4
1  img_2     4
2  img_3     2
3  img_4     1
4  img_5     3
(6347, 2)
```

```
✓ [56] 1 submission.to_csv('/content/drive/MyDrive/submission.csv', index = False)
```

**THANK YOU FOR YOUR
ATTENTION**